

本地无法计算的内容！

Fabian Kuhn
计算机科学系
苏黎世联邦理工学院
瑞士苏黎世 8092
kuhn@inf.ethz.ch

Thomas Moscibroda
计算机科学系
苏黎世联邦理工学院
瑞士苏黎世 8092
moscitho@inf.ethz.ch

Roger Wattenhofer
计算机科学系
苏黎世联邦理工学院
瑞士苏黎世 8092
wattenhofer@inf.ethz.ch

ABSTRACT

我们给出了最小顶点覆盖（MVC）及相关问题（如最小支配集（MDS））分布式近似的时间下界。在 k 通信轮次中，MVC 和 MDS 只能被近似到因子 $\Omega(n^{c/k^2}/k)$ 和 $\Omega(\Delta^{1/k}/k)$ ，其中 c 为常数， n 和 Δ 分别表示图中的节点数和最大度数。要达到常数甚至仅对数多项式近似比所需的轮次至少为 $\Omega(\sqrt{\log n / \log \log n})$ 和 $\Omega(\log \Delta / \log \log \Delta)$ 。通过简单归约，后者的下界同样适用于最大匹配和最大独立集的构造。

主要证明目标

类别与主题描述符

F.2.2 [算法分析与问题复杂性]

ity]: 非数值算法与问题——离散结构上的计算;

G.2.2 [离散数学]: 图论——图算法;

G.2.2 [离散数学]: 图论——网络问题

通用术语

算法, 理论

关键词

近似难度, 分布式算法, 支配集, 局部性, 下界, 最大独立集, 最大匹配, 顶点覆盖

*本文所述工作（部分）得到了 Hasler 基金会（瑞士伯尔尼）以及国家移动信息与通信系统能力研究中心（NCCR-MICS）的支持，该中心由瑞士国家科学基金会资助，资助编号为 5005-67322。

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

PODC'04, July 25–28, 2004, St. John's, Newfoundland, Canada.
版权所有 2004 ACM 1-58113-802-4/04/0007 ...\$5.00

1. 引言与相关工作

什么可以在本地计算？Naor 和 Stockmayer [18] 在十多年前向分布式计算界提出了这个关键问题，尽管此后局部性的代价一直困扰着研究者，但成果寥寥 [15]。局部性的重要性源于仅基于局部信息实现全局目标的愿望。这不仅是在开发快速分布式算法时面临的关键挑战之一，还能通过检查局部条件来提高容错性并检测非法全局配置 [1]。网络技术的近期进展和不断增长的分布式系统推动了对局部性问题深入理解的需求。在本文中，我们研究了经典消息传递模型中局部性的影响，其中分布式系统被建模为图：节点代表处理器，两个节点当且仅当在图中共有一条边时才能通信。

什么算消息传递模型

我们针对几个传统图论问题给出了下界，从最小顶点覆盖（MVC）开始。图 $G = (V, E)$ 的顶点覆盖是节点子集 $V' \subseteq V$ ，使得对于每条边 $(u, v) \in E$ ，两个关联节点 u, v 中至少有一个属于 V' 。寻找基数最小的顶点覆盖被称为 MVC 问题。

在局部算法中，节点只允许与 G 中的直接邻居通信。经过几轮通信后，它们需要得出全局解，例如 MVC 的良好近似。直观上，MVC 可能被认为非常适合局部算法：节点应能通过与其邻居通信几次来决定是否加入顶点覆盖。非常远的节点似乎对此决策是多余的。存在一个简单的贪心算法，能在全局设置中以因子 2 近似 MVC，这进一步增加了对高效局部算法的期望。

然而，令我们惊讶的是，并不存在这样的算法。在本文中，我们证明在 k 轮通信中，MVC 只能被近似到因子 $\Omega(n^{c/k^2}/k)$ ，其中常数 c 大于 $1/4$ 。这意味着为了达到常数甚至多对数近似比，需要运行时间 $\Omega(\sqrt{\log n / \log \log n})$ 。当结果依赖于最大度 Δ 而非节点数 n 时，近似比为 $\Omega(\Delta^{1/k}/k)$ ，而获得多对数或常数近似所需的时间为 $\Omega(\log \Delta / \log \log \Delta)$ 。MVC 的结果可以扩展到最小支配集（MDS）和其他分布式覆盖问题。此外，时间

常数 MVC 近似的时间下界也适用于最大匹配和最大独立集的构造。由于所有下界即使在消息无界和完全同步的情况下也成立，因此这些下界是局部性限制的真正结果，而不仅仅是拥塞、异步或消息大小有限的副作用。

此外，我们的部分下界在许多情况下几乎是紧的。在 k 轮内，MVC 可近似到因子 $O(\Delta^{1/k})$ [10]。为获得多对数近似，需设 $k = O(\log \Delta / \log \log \Delta)$ ；对于常数近似，轮数为 $k = O(\log \Delta)$ 。因此，对于多对数（及更差）近似比，我们的下界是紧的；而对于常数近似算法，其紧度至多差因子 $O(\log \log \Delta)$ 。对于 MDS 问题，最新结果表明，在 k 轮内，相应线性规划可近似到因子 $\Delta^{c/\sqrt{k}}$ ，而 MDS 本身可实现近似比 $\Delta^{c/\sqrt{k}} \ln \Delta$ [9, 10]。若消息大小和局部计算无界，MDS LP 和 MDS 可分别近似到因子 $O(n^{1/k})$ 和 $O(n^{1/k} \ln \Delta)$ [10]。易验证 MDS 的下界对分数 LP 版本同样成立。最大匹配和最大独立集的最优已知算法需要时间 $O(\log n)$ [8, 17]。

以上是该论文证明的内容

Linial [15] 开创性的下界表明，Cole 和 Vishkin [2] 的非均匀 $O(\log^* n)$ 着色算法在环上渐近最优。该下界被研究者视为分布式算法理论的基础性进展。针对不同分布式计算模型，存在其他下界 [6, 11]，最著名的是网络图 [3, 4, 16, 20] 的最小生成树 (MST) 构造问题。除 [3] 外，MST 下界适用于消息大小有界的模型。[4] 将 [16, 20] 的下界扩展到近似算法。据我们所知，这是此前唯一关于分布式近似难度的下界。

Linial 的下界 [15] 基于分布式计算的果蝇——环网络。对于 MVC 问题，环等高度对称图常具有常数近似比的直接解法。例如，在任何 δ 正则图中，包含所有顶点的顶点覆盖算法已是 MVC 的 2-近似：每个顶点最多覆盖 δ 条边，图有 $n\delta/2$ 条边，因此最小顶点覆盖至少需要 $n/2$ 个顶点。另一方面，非对称图也常享有常数时间算法。在树中，选择所有内部节点可得 2-近似。节点度数也存在类似权衡：若最大度数低（常数），可容忍选择所有节点，从而自然获得良好（常数）近似；若存在高度数节点，图直径小，少数通信轮次即可让所有节点获知全图信息。我们需要构造一个既不过于对称也不过于不对称、且节点度数多样的图！在分布式计算中，具备这些“非性质”的图并不多见。

我们下界的证明基于永恒的不可区分性论证 [7, 12]。在 k 轮通信中

在通信中，网络节点只能收集最多 k 跳范围内的节点信息，因此只能利用这些信息来确定计算结果。特别地，我们证明经过 k 轮通信后，两个相邻节点看到的图拓扑完全相同；通俗地说，两个邻居都有同等资格加入顶点覆盖。然而，在我们的示例图中，选择错误的邻居将导致灾难性后果。

本文其余部分组织如下。第2节描述计算模型。下界在第3节中构建。在第4节中，我们展示如何将MVC下界扩展到最小支配集 (MDS)、最大匹配 (MM) 和最大独立集 (MIS)。第5节总结全文。

2. MODEL 模型定义

我们考虑经典的消息传递模型，该模型由点对点通信网络组成，由无向图 $G = (V, E)$ 描述。节点对应处理器，边表示它们之间的通信信道。在一轮通信中，网络图中的每个节点可以向每个邻居发送任意长的消息。局部计算是免费的，且初始时节点对网络图一无所知。它们只知道自己的唯一标识符。¹ 在 k 轮通信后，节点 v 可以收集距离 v 的 $\mathcal{T}_{v,k}$ 最多 k 跳的所有节点的ID和互连关系，定义为 v 在这 k 轮后看到的拓扑，即 $\mathcal{T}_{v,k}$ 是由 v 的 k 邻域导出的图，其中恰好距离为 k 的节点之间的边被排除。 $\mathcal{T}_{v,k}$ 的标记（即ID分配）记为 $\mathcal{L}(\mathcal{T}_{v,k})$ 。

由于消息大小无限制，确定性算法在时间 k 内能做的最好事情是收集其 k 邻域并基于 $(\mathcal{T}_{v,k}, \mathcal{L}(\mathcal{T}_{v,k}))$ 做出决策。换句话说，确定性分布式算法可以视为将 $(\mathcal{T}_{v,k}, \mathcal{L}(\mathcal{T}_{v,k}))$ 映射到可能输出的函数。对于随机算法， v 的结果还取决于 $\mathcal{T}_{v,k}$ 中节点计算的随机性。

所提出的模型与[15]和教科书[19]中使用的模型一致。在证明局部计算的下界时，这是最强的可能模型，因为它完全聚焦于分布式问题的局部性，并抽象了分布式算法设计中出现的其他问题（例如，需要小消息、快速局部计算等）。这保证了我们的下界是局部性的真实结果。

二分子图：将所有的节点分成两组，所有的边都只连接左右两组，组内没有边

3. 下界

我们首先给出证明的概要。基本思想是构造一个图 $G_k = (V, E)$ ，对于每个正整数 k ，该图包含一个二分子图 S ，其节点集为 $C_0 \cup C_1$ ，边集为 $C_0 \times C_1$ ，如图1所示。集合 C_0 由 n_0 个节点组成，每个节点在 C_1 中有 δ_0 个邻居。 C_1 中的 $n_0 \cdot \frac{\delta_0}{\delta_1}$ 个节点每个在 C_0 中有 $\delta_1, \delta_1 > \delta_0$ 个邻居。目标是构造 G_k ，使得 $v \in S$ 中的所有节点在距离 k 内看到相同的拓扑 $\mathcal{T}_{v,k}$ 。在全局最优解中， S 的所有边可能被 C_1 中的节点覆盖，因此 C_0 中无需节点加入顶点覆盖。

¹ 我们的结果适用于任何可能的 ID 空间，包括标准情况，即 ID 为数字 $1, \dots, n$ 。

然而，在局部算法中，节点是否加入顶点覆盖的决定仅取决于其局部视图，即对 $(\mathcal{T}_{v,k}, \mathcal{L}(\mathcal{T}_{v,k}))$ 。我们证明，由于 S 中相邻节点看到相同的 $\mathcal{T}_{v,k}$ ，每个算法必须将 C_0 中的大量节点加入其顶点覆盖，才能得到可行解。换句话说，我们构造了一个图，其中两个相邻节点之间的对称性无法在 k 轮通信内打破。这导致次优的局部决策，从而得到次优的近似比。在整个证明中，我们将使用 C_0 和 C_1 来表示二分子图 S 的两个集合。

我们的证明组织如下。 G_k 的结构在 3.1 节中定义。在 3.2 节中，我们展示如何构造无小环的 G_k ，确保每个节点在距离 k 内看到一棵树。3.3 节证明 C_0 和 C_1 中相邻节点具有相同的视图 $\mathcal{T}_{v,k}$ ，最后，3.4 节推导下界。

3.1 簇树

图 $G_k = (V, E)$ 的节点可以分组为不相交的集合，这些集合以二部图的形式相互连接。我们将这些不相交的节点集称为簇。

我们使用带有双标记弧 $\ell: \mathcal{A} \rightarrow \mathbb{N} \times \mathbb{N}$ 的有向树 $CT_k = (\mathcal{C}, \mathcal{A})$ 来定义 G_k 的结构。我们将 CT_k 称为聚类树，因为每个顶点 $C \in \mathcal{C}$ 代表 G_k 中节点的一个聚类。聚类 $|C|$ 的大小是该聚类包含的节点数。一条带有 $\ell(a) = (\delta_C, \delta_D)$ 的弧 $a = (C, D) \in \mathcal{A}$ 表示聚类 C 和 D 以二部图的形式连接，使得每个节点 $u \in C$ 在 D 中有 δ_C 个邻居，每个节点 $v \in D$ 在 C 中有 δ_D 个邻居。由此可得 $|C| \cdot \delta_C = |D| \cdot \delta_D$ 。如果一个聚类仅与另一个聚类相邻，我们称之为叶聚类，否则称之为内部聚类。

定义 3.1. 聚类树 CT_k 递归定义如下：

$$\begin{aligned} CT_1 &:= (C_1, \mathcal{A}_1), \quad C_1 := \{C_0, C_1, C_2, C_3\} \\ \mathcal{A}_1 &:= \{(C_0, C_1), (C_0, C_2), (C_1, C_3)\} \\ \ell(C_0, C_1) &:= (\delta_0, \delta_1), \quad \ell(C_0, C_2) := (\delta_1, \delta_2), \\ \ell(C_1, C_3) &:= (\delta_0, \delta_1) \end{aligned}$$

给定 CT_{k-1} ，我们通过两步得到 CT_k ：

• 对于每个内部聚类 C_i ，添加一个新的叶聚类 C'_i ，其 $\ell(C_i, C'_i) := (\delta_k, \delta_{k+1})$ 。

对于 CT_{k-1} 中每个带有 $(C'_i, C_i) \in \mathcal{A}$ 和 $\ell(C'_i, C_i) = (\delta_p, \delta_{p+1})$ 的叶聚类 C'_i ，为 $j = 0 \dots k, j \neq p+1$ 添加 $k-1$ 个新的叶聚类 C'_j ，其 $\ell(C_i, C'_j) := (\delta_i, \delta_{j+1})$ 。

此外，我们为所有 CT_k 定义 $|C_0| = n_0$ 。

图1展示了 CT_2 。阴影子图对应 CT_1 。每条弧 $a \in \mathcal{A}$ 的标签形式为 $\ell(a) = (\delta_l, \delta_{l+1})$ ，其中 $l \in \{0, \dots, k\}$ 为某个值。此外，设定 $|C_0| = n_0$ 唯一确定了所有其他簇的大小。为简化后续对簇树的研究，我们需要两个额外定义。簇的层级是其在簇树中到 C_0 的距离（参见图1）。簇 C 的深度是其到以 C 为根的子树中最远叶节点的距离。因此，簇的深度加一等于 C 对应子树的高度。在图1的示例中， C_0, C_1, C_2 和 C_3 的深度分别为3、2、1和1。

清楚概念：簇、二部图、双标记弧、 CT_k 、聚类、叶聚类、内部聚类、簇的层级、簇C的深度

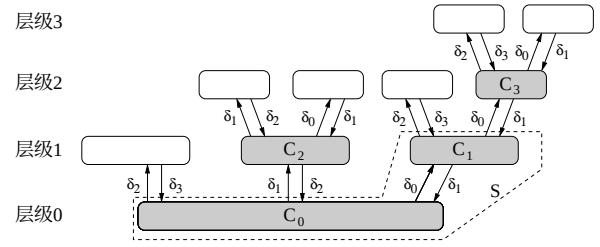


图1：簇树 CT_2 。

注意， CT_k 描述了 G_k 的通用结构，即为每个节点定义了每个簇中的邻居数量。然而， CT_k 并未指定实际的邻接关系。在下一小节中，我们将展示可以构造 G_k ，使得每个节点的视图是一棵树。

即明确哪些节点之间有边

3.2 下界图 把簇树蓝图具化成真实的图，保证围长大于2k

在3.3小节中，我们将证明 C_0 和 C_1 中节点看到的拓扑结构是相同的。如果每个节点的拓扑是一棵树（而非一般图），这一任务将大大简化，因为我们无需担心环的存在。图 G 的围长，记为 $g(G)$ ，是 G 中最短环的长度。我们希望构造围长至少为 $2k+1$ 的 G_k ，使得在 k 轮通信后，所有节点看到的都是一棵树。鉴于大 k 下 G_k 的结构复杂性，构造大围长的 G_k 并非易事。我们提出的解决方案基于文献[13]中图族 $D(r, q)$ 的构造。对于给定的 r 和 q ， $D(r, q)$ ，定义了一个具有 $2q^r$ 个节点和围长 $g(D(r, q)) \geq r+5$ 的二部图。特别地，我们证明对于适当的 r 和 q ，通过删除 $D(r, q)$ 的部分边，可以得到 G_k 的一个实例。下面，我们将介绍理解结果所需的 $D(r, q)$ 细节。感兴趣的读者可参阅文献[13]。

对于整数 $r \geq 1$ 和素数幂 q ， $D(r, q)$ ，定义了一个二部图，其节点集为 $P \cup L$ ，边集为 $E_D \subset P \times L$ 。 P 和 L 的节点由有限域 $\mathbb{F}_q$ 上的 r 维向量标记，即 $P = L = \mathbb{F}_q^r$ 。根据 [13]，我们将向量 $p \in P$ 记为 (p) ，将向量 $l \in L$ 记为 $[l]$ 。 (p) 和 $[l]$ 的分量如下书写（对于 $D(r, q)$ ，向量投影到前 r 个坐标上）：

$$(p) = (p_1, p_{1,1}, p_{1,2}, p_{2,1}, p_{2,2}, p_{2,3}, p_{3,2}, \dots, p_{i,i}, p'_{i,i}, p_{i,i+1}, p_{i+1,i}, \dots) \quad (1)$$

$$[l] = [l_1, l_{1,1}, l_{1,2}, l_{2,1}, l_{2,2}, l'_{2,2}, l_{2,3}, l_{3,2}, \dots, l_{i,i}, l'_{i,i}, l_{i,i+1}, l_{i+1,i}, \dots] \quad (2)$$

注意， (p) 和 $[l]$ 的分量编号有些混乱，这是为了简化以下方程组。两个节点 (p) 和 $[l]$ 之间存在一条边，当且仅当以下条件的前 $r-1$ 个成立（对于 $i = 2, 3, \dots$ ）。

$$\begin{aligned} l_{1,1} - p_{1,1} &= l_1 p_1 \\ l_{1,2} - p_{1,2} &= l_{1,1} p_1 \\ l_{2,1} - p_{2,1} &= l_1 p_{1,1} \\ l_{i,i} - p_{i,i} &= l_1 p_{i-1,i} \\ l'_{i,i} - p'_{i,i} &= l_{i,i-1} p_1 \\ l_{i,i+1} - p_{i,i+1} &= l_{i,i} p_1 \\ l_{i+1,i} - p_{i+1,i} &= l_1 p'_{i,i} \end{aligned} \quad (3)$$

在 [13] 中, 证明了对于奇数 $r \geq 3, D(r, q)$, 其围长至少为 $r + 5$ 。此外, 如果固定节点 u 和邻居 v 的一个坐标, 则 v 的其余坐标唯一确定。这在下一个引理中具体化。

引理 3.2. 对于所有 $(p) \in P$ 和 $l_1 \in \mathbb{F}_q$, 恰好存在一个 $[l] \in L$, 使得 l_1 是 $[l]$ 的第一个坐标, 并且 (p) 和 $[l]$ 在 $D(r, q)$ 中由一条边连接。类似地, 如果固定 $[l] \in L$ 和 $p_1 \in \mathbb{F}_q$, 则 $[l]$ 的邻居 (p) 唯一确定。

证明。 (3) 的前 $r - 1$ 个方程定义了关于 $[l]$ 未知坐标的线性系统。如果按给定顺序书写方程和变量, 则所得线性方程组对应的矩阵是一个下三角矩阵, 且对角线元素非零。因此, 该矩阵满秩, 根据 (有限) 域的基本法则, 解是唯一的。完全相同的论证适用于引理的第二个结论。

先构建一个符合簇树定义的 G_k' , 然后将这个图扩展入 $D(r, q)$ □

我们现在准备构造具有大围长的 G_k 。我们从聚类树的任意实例 G'_k 开始, 该实例可能具有最小可能的围长 4。关于 G'_k 构造的详细说明推迟到第 3.4 小节。目前, 我们仅假设 G'_k 存在。 G_k 和 G'_k 都是二分图, 其中奇数层簇在一侧, 偶数层簇在另一侧。设 m 为 G'_k 两个分区中较大分区的节点数。我们选择 q 为大于或等于 m 的最小素数幂。在 G'_k 的两个分区 $V_1(G'_k)$ 和 $V_2(G'_k)$ 中, 我们用元素 $c(v) \in \mathbb{F}_q$ 对所有节点 v 进行唯一标记。

如前所述, G_k 被构造为 $D(r, q)$ 的子图, 针对适当的 r 和 q 。我们按上述方式选择 q , 并设置 $r = 2k - 4$ 使得 $g(D(r, q)) \geq 2k + 1$ 。设 $(p) = (p_1, \dots)$ 和 $[l] = [l_1, \dots]$ 为 $D(r, q)$ 的两个节点。 (p) 和 $[l]$ 在 G_k 中由一条边连接, 当且仅当它们在 $D(r, q)$ 中相连, 并且节点 $u \in V_1(G'_k)$ 和 $v \in V_2(G'_k)$ 之间存在一条边, 使得 $c(u) = p_1$ 和 $c(v) = l_1$ 。最后, 没有关联边的节点从 G_k 中移除。

引理 3.3. 按上述方式构造的图 G_k 是一个聚类树, 具有与 G'_k 中相同的度数 δ_i 。 G_k 最多有 $2mq^{2k-5}$ 个节点, 围长至少为 $2k + 1$ 。

证明。围长直接由构造得出; 删除边不会产生环。

对于簇之间的度数, 考虑 G'_k 中两个相邻的簇 $C'_i \subset V_1(G'_k)$ 和 $C'_j \subset V_2(G'_k)$ 。在 G_k 中, 每个节点被替换为 q^{2k-5} 个新节点。簇 C_i 和 C_j 由所有节点 (p) 和 $[l]$ 组成, 这些节点的第一个坐标分别等于 C'_i 和 C'_j 中节点的标签。设 C'_i 中的每个节点在 C'_j 中有 δ_α 个邻居, 且 C'_j 中的每个节点在 C'_i 中有 δ_β 个邻居。根据引理 3.2, C_i 中的节点在 C_j 中有 δ_α 个邻居, C_j 中的节点在 C_i 中也有 δ_β 个邻居。 □

注。在 [14] 中, 已证明 $D(r, q)$ 是不连通的, 且由至少 $q^{\lfloor \frac{r+2}{4} \rfloor}$ 个同构组件组成, 作者称这些组件为 $CD(r, q)$ 。显然, 这些组件也是有效的簇树, 我们可以使用其中一个进行分析。由于我们的渐近结果不受此观察影响, 我们继续使用如上构造的 G_k 。

3.3 视图的等价性

在本小节中, 我们证明簇 C_0 和 C_1 中两个相邻节点具有相同的视图, 即在距离 k 内, 它们看到完全相同的拓扑 $\mathcal{T}_{v,k}$ 。考虑一个节点 $v \in G_k$ 。鉴于 v 的视图是一棵树, 我们可以通过递归地跟随 v 的所有邻居来推导其视图树。证明主要基于以下观察: 对应的子树出现在两个节点的视图树中。

设 C_i 和 C_j 是 CT_k 中通过 $\ell(C_i, C_j) = (\delta_i, \delta_{i+1})$ 连接的相邻簇, 即 C_i 中的每个节点在 C_j 中有 δ_i 个邻居, C_j 中的每个节点在 C_i 中有 δ_{i+1} 个邻居。在遍历节点的视图树时, 我们说我们从簇 C_i (或 C_j) 通过链路 δ_i (或 δ_{i+1}) 进入簇 C_j (或 C_i)。此外, 我们做出以下定义:

- M_i 是通过链路 δ_i 进入簇 C_0 时看到的子树。
- $B_{i,d,\lambda}$ 是通过链路 δ_i 进入簇 $C \in \mathcal{C} \setminus \{C_0\}$ 时看到的子树, 其中 C 位于层级 λ , 深度为 d 。

定义 3.5. 当从层级 $\lambda - 1$ ($\lambda + 1$) 的簇进入子树 $B_{i,d,\lambda}$ 时, 我们记作 $B_{i,d,\lambda}^\uparrow$ ($B_{i,d,\lambda}^\downarrow$)。谓词 $\neg$ 在 $B_{i,d,\lambda}^\downarrow$ 中表示, 进入该子树的链路标签不是 δ_i , 而是 $\delta_i - 1$ 。

谓词 $\neg$ 在以下情况是必要的: 从 C_i 进入 C_j 后, 立即通过链路 δ_i 返回 C_i 。在视图树中, 用于进入 C_j 的边连接当前子树与其父节点。因此, 该边不再可用, 只剩下 $\delta_i - 1$ 条边可返回 C_i 。谓词 $\uparrow$ 和 $\downarrow$ 描述了从哪个“方向”进入簇。由于 C_0 和 C_1 中节点的视图树必须完全一致才能使证明成立, 我们不能忽略这些确实繁琐的细节。

以下示例应能阐明各种定义。此外, 可参考附录中图 3 的 G_3 示例。

示例 3.6. 考虑 G_1 。设 V_{C_0} 和 V_{C_1} 分别表示 C_0 和 C_1 中节点的视图树:

$$\begin{aligned} V_{C_0} &= B_{0,1,1}^\uparrow \cup B_{1,0,1}^\downarrow & V_{C_1} &= B_{0,0,2}^\uparrow \cup M_1 \\ B_{0,1,1}^\uparrow &= B_{0,0,2}^\uparrow \cup M_1^\neg & B_{0,0,2}^\uparrow &= B_{1,1,1}^\downarrow \\ B_{1,0,1}^\downarrow &= M_2^\neg & M_1 &= B_{0,1,1}^\neg \cup B_{1,0,1}^\uparrow \\ \dots & & \dots & \end{aligned}$$

我们通过给出描述视图树中某点所见子树的一组规则来开始证明。我们称这些规则为推导规则, 因为它们允许我们通过机械地应用给定子树的匹配规则来推导节点的视图树。

引理 3.7. 以下推导规则在 G_k 中成立:

$$\begin{aligned} M_i &= \bigcup_{\substack{j=0 \dots k \\ j \neq i-1}} B_{j,k-j,1}^\uparrow \cup B_{i-1,k-i+1,1}^\neg \\ B_{i,d,1}^\uparrow &= F_{\{i+1\}} \cup D_{\{\}} \cup M_{i+1}^\neg \\ B_{i,d,1}^\downarrow &= F_{\{i-1,k-d+1\}} \cup D_{\{\}} \cup M_{k-d+1} \cup B_{i-1,d-1,2}^\neg \\ B_{i,d,\lambda}^\uparrow &= F_{\{i+1\}} \cup D_{\{i+1\}} \cup B_{i+1,d+1,\lambda-1}^\neg \end{aligned}$$

其中 F 和 D 定义为

$$F_W := \bigcup_{\substack{j=0 \dots k-d+1 \\ j \notin W}} B_{j,d-1,\lambda+1}^\uparrow$$

$$D_W := \bigcup_{\substack{j=k-d+2 \dots k \\ j \notin W}} B_{j,k-j,\lambda+1}^\uparrow$$

证明。我们首先证明 M_i 的推导规则。从示例 3.6 中可看出该规则对 $k=1$ 成立。归纳步骤中，我们按定义 3.1 从 CT_k 构建 CT_{k+1} 。 $M^{(k)}$ 是一个内部簇，因此添加了一个新簇 $B_{k+1,0,1}$ 。所有其他子树的深度增加 1， $M^{(k+1)} := \bigcup_{j=0 \dots k+1} B_{j,k-j,1}^\uparrow$ 随之成立。如果通过链路 δ_i 进入 $M^{(k+1)}$ ，则只剩下 $\delta_{i-1} - 1$ 条边可返回进入 C_0 的簇。因此，链路 δ_{i-1} 具有 $\neg$ 谓词。

其余规则遵循相同思路。设 C_i 是一个簇，其入口链接为 δ_i ，首次创建于 CT_r ， $r < k$ （注意在 $CT_k, r = k - d$ 中成立，因为每个子树在每“轮”中深度增加一）。根据定义 3.1 的第二条构建规则， r 个新的相邻簇（子树）在 CT_{r+1} 中创建。更准确地说，为除 δ_i 外的所有入口链接 $\delta_0 \dots \delta_r$ 创建一个新簇。我们称这些子树为固定深度子树 F 。若根为 C_i 的子树在 CT_k 中深度为 d ，则固定深度子树的深度为 $d - 1$ 。在每个 $CT_{r'}, r' \in \{r+2, \dots, k\}$ ， C_i 中是一个内部簇，因此创建一个入口链接为 $\delta_{r'}$ 的新相邻簇。我们称这些子树为递减深度子树 D 。在 CT_k 中，每个此类子树已增长至深度 $k - r'$ 。

现在我们关注三条规则之间的差异。这些差异源于对第 1 层的特殊处理，以及谓词 $\uparrow$ 和 $\downarrow$ 。在 $B_{i,d,1}^\uparrow$ 中，链接 δ_{i+1} 返回 C_0 ，但在视图树中仅包含 $\delta_{i+1} - 1$ 条边。在 $B_{i,d,1}^\downarrow$ 中，我们必须考虑两种特殊情况。第一种是到 C_0 的链接。对于第 1 层上入口链接为（来自 C_0 ） i 的簇，等式 $k = d + i$ 成立，因此到 C_0 的链接为 δ_{k-d+1} ，从而 M_{k-d+1} 成立。其次，我们写 $B_{i-1,d-1,2}^\uparrow$ ，因为返回我们来源簇的边少了一条。（注意，由于我们从更高层进入当前簇，返回来源的链接是 δ_{i-1} ，而非 δ_{i+1} ）。最后在 $B_{i,d,\lambda}^\uparrow$ 中，我们再次需要特殊处理返回链接 δ_{i+1} 。

注意， $B_{i,d,\lambda}^\downarrow$ 的通用推导规则缺失，因为证明中不需要它。□

接下来，我们定义 r -相等的概念。直观上，若两个视图树在距离 r 内具有相同拓扑结构，则它们 r -相等。

定义 3.8。设 $V_1 = \bigcup_{i=0 \dots k} b_i$ 和 $V_2 = \bigcup_{i=0 \dots k} b'_i$ 为视图树； b_i 和 b'_i 是通过链接 δ_i 进入的子树。那么，若所有对应子树 $(r-1)$ -相等，则 V_1 和 V_2 是 r -相等的。

$$V_1 \stackrel{r}{=} V_2 \iff b_i \stackrel{r-1}{=} b'_i, \forall i \in \{0, \dots, k\}.$$

此外，所有子树都是 0-等价的：对于所有 i, i', d, d', λ ，有 $B_{i,d,\lambda} = B_{i',d',\lambda'}$ 和 $B_{i,d,\lambda} = M_{i'}$ ，以及 λ' 。

利用 r -等价的概念，现在可以轻松定义我们实际需要证明的内容。我们将证明在 G_k 中，

$V_{C_0} \stackrel{k}{=} V_{C_1}$ 成立。这等价于证明 C_0 中的每个节点在距离 k 内看到的拓扑结构与其在 C_1 中的邻居完全相同。我们现在将建立几个辅助引理。

引理 3.9。设 β 和 β' 为子树集合，并设

$$V_{v_1} = B_{i,d,x}^\uparrow \cup \beta \text{ 和 } V_{v_2} = B_{i,d,y}^\uparrow \cup \beta'$$

那么，对于所有 x 和 y

$$V_{v_1} \stackrel{r}{=} V_{v_2} \iff \beta \stackrel{r-1}{=} \beta'.$$

证明。假设 V_{v_1} 和 V_{v_2} 子树的根分别为 C_i 和 C_j 。这些子树具有相同的深度和入口链接，因此它们以相同方式生长。因此，所有不返回簇 C_i 和 C_j 的路径必须相同。进一步，考虑所有在 s 跳后通过链接 δ_{i+1} 返回 C_i 和 C_j 的路径。在这些 s 跳之后，它们返回原始簇并看到视图 V_{v_1} 和 V_{v_2} ，与 V_{v_1} 和 V_{v_2} 仅在 $\neg$ 谓词的位置上不同。这并不影响 β 和 β' ，因此，

$$V_{v_1} \stackrel{r}{=} V_{v_2} \iff V_{v_1} \stackrel{r-s}{=} V_{v_2}' \wedge \beta \stackrel{r-1}{=} \beta', s > 1.$$

同样的论证可以重复直到 $r - s = 0$ ，并且由于

$$V_{v_1}' = V_{v_2}', \text{ 引理得证。} \quad \square$$

引理 3.10。设 β 和 β' 为子树集合， $V_{v_1} = B_{i,d,x}^\uparrow \cup \beta$ 和 $V_{v_2} = B_{i',d',y}^\uparrow \cup \beta'$ 为两个视图树。那么，对于所有 x 和 y ，以及所有 $r \leq \min(d, d')$ ，

$$V_{v_1} \stackrel{r}{=} V_{v_2} \iff \beta \stackrel{r-1}{=} \beta'.$$

证明。不失一般性，我们假设 $d' \leq d$ 。在 G_k 的构建过程中， V_{v_1} 和 V_{v_2} 的根簇分别在步骤 $k - d$ 和 $k - d'$ 中创建。根据定义 3.1，所有深度为 $d^* < d'$ 的子树在两个视图树中以相同方式生长。 V_{v_2} 中剩余的子树均在步骤 $k - d' + 1$ 中创建，深度为 $d' - 1$ 。 V_{v_1} 中对应的子树至少具有相同深度，因此每对对应子树是 $(d' - 1)$ 等价的。由此可得，对于 $r \leq \min(d, d')$ ，子树 $B_{i,d,x}^\uparrow$ 和 $B_{i',d',y}^\uparrow$ 在距离 r 内相同。使用与引理 3.9 相同的论证即可完成证明。□

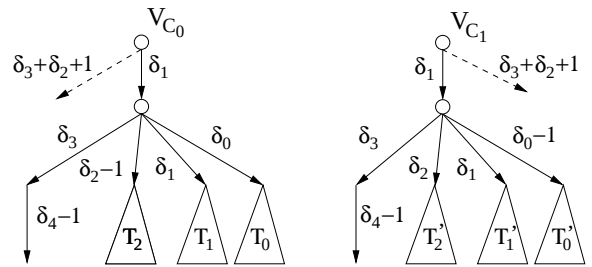


图 2：使用链接 δ_1 时在 G_3 中看到的视图树 V_{C_0} 和 V_{C_1} 。

图 2 显示了 G_3 中节点 C_0 和 C_1 的视图树的一部分。该图显示，由于 -1 的位置不同，带有链接 δ_0 和 δ_2 的子树无法直接相互匹配。事实证明，这种固有差异出现在我们定理的每一步中。然而，以下引理表明，子树 T_0 、 T_2 (T'_0 和 T'_2) 在所需距离内相等，并且

因此，节点无法区分它们。正是我们簇树的这一关键特性，使得我们能够在链接 δ_i 和 δ_{i+2} 之间“移动” $\neg$ 谓词，并推导出主要定理。

引理 3.11. 设 β 和 β' 为子树集合，并设 V_{v_1} 和 V_{v_2} 定义为

$$\begin{aligned} V_{v_1} &= M_i^\neg \cup B_{i-2,k-i,2}^\uparrow \cup \beta \\ V_{v_2} &= M_i \cup B_{i-2,k-i,2}^{\uparrow,\neg} \cup \beta'. \end{aligned}$$

那么，对于所有 $i \in \{2, \dots, k\}$,

$$V_{v_1} \stackrel{k-i}{=} V_{v_2} \Leftarrow \beta \stackrel{k-i-1}{=} \beta'.$$

证明。同样，我们利用引理 3.7 来证明 M_i 和 $B_{i-2,k-i,2}^\uparrow$ 是 $(k-i-1)$ 等价的。由于这两个子树不可区分且 $\neg$ 谓词的位置无关紧要，结论由此得证。

$$\begin{aligned} M_i &= \bigcup_{\substack{j=0\dots k \\ j \neq i-1}} B_{j,k-j,1}^\uparrow \cup B_{i-1,k-i+1,1}^{\uparrow,\neg} \\ B_{i-2,k-i,2}^\uparrow &= \bigcup_{\substack{j=0\dots i+1 \\ j \neq i-1}} B_{j,k-i-1,3}^\uparrow \cup \bigcup_{j=i+2\dots k} B_{j,k-j,3}^\uparrow \\ &\quad \cup B_{i-1,k-i+1,1}^{\downarrow,\neg} \end{aligned}$$

对于 $j = \{0, \dots, i-2, i, \dots, k\}$ ，根据引理 3.9 和 3.10，所有子树都相等。还需证明 $B_{i-1,k-i+1,1}^\uparrow = B_{i-1,k-i+1,1}^\downarrow$ 。为此，我们将 $B_{i-1,k-i+1,1}^\uparrow$ 和 $B_{i-1,k-i+1,1}^\downarrow$ 代入引理 3.7，并使用推导规则证明它们的相等性。设 β 定义为 $\beta := F_{\{i-2,i\}} \cup D_{\{\}} \cup \beta$ 。

$$\begin{aligned} B_{i-1,k-i+1,1}^\uparrow &= F_{\{i\}} \cup D_{\{\}} \cup M_i^\neg \\ &= B_{i-2,k-i,2}^\uparrow \cup M_i^\neg \cup \beta \\ B_{i-1,k-i+1,1}^\downarrow &= F_{\{i-2,i\}} \cup D_{\{\}} \cup M_i \cup B_{i-2,k-i,2}^{\uparrow,\neg} \\ &= B_{i-2,k-i,2}^{\uparrow,\neg} \cup M_i \cup \beta \end{aligned}$$

同样，如果 M_i 和 $B_{i-2,k-i,2}^\uparrow$ 是 $(k-i-3)$ 等价的，由于子树不可区分，我们可以移动 $\neg$ 谓词。因此，我们必须证明 $M_i \stackrel{k-i-3}{=} B_{i-2,k-i,2}^\uparrow$ 。在证明中，我们已将

$V_{v_1} \stackrel{k-i}{=} V_{v_2}$ 逐步简化为一个等式条件递减的表达式，即

$$\begin{aligned} V_{v_1} \stackrel{k-i}{=} V_{v_2} &\Leftarrow M_i \stackrel{k-i-1}{=} B_{i-2,k-i,2}^\uparrow \\ &\Leftarrow B_{i-1,k-i+1,1}^\uparrow \stackrel{k-i-2}{=} B_{i-1,k-i+1,1}^\downarrow \\ &\Leftarrow M_i \stackrel{k-i-3}{=} B_{i-2,k-i,2}^\uparrow. \end{aligned}$$

此过程可继续，直到

$$B_{i-1,k-i+1,1}^\uparrow \stackrel{0}{=} B_{i-1,k-i+1,1}^\downarrow \quad \text{or} \quad M_i \stackrel{0}{=} B_{i-2,k-i,2}^\uparrow$$

这总是成立。 $\square$

最后，我们准备证明主要定理。

定理 3.12. 考虑图 G_k 。设 V_{C_0} 和 V_{C_1} 分别为簇 C_0 和 C_1 中两个相邻节点的视图树。那么， $V_{C_0} = V_{C_1}$ 。

证明。初始时， C_0 中的每个节点看到子树 M_* ， C_1 中的每个节点看到 $B_{*,k,1}$ （* 表示该子树尚未在任何链接上进入）：

$$\begin{aligned} V_{C_0} &: M_* = \bigcup_{j=0\dots k} B_{j,k-j,1}^\uparrow \\ V_{C_1} &: B_{*,k,1} = \bigcup_{\substack{j=0\dots k \\ j \neq 1}} B_{j,k-j,2}^\uparrow \cup M_1. \end{aligned}$$

由此得出 $V_{C_0} \triangleq V_{C_1} \Leftarrow B_{1,k-1,1}^{\uparrow,k-1} M_1$ ，因为根据引理 3.9，所有其他子树都是 $(k-1)$ 等价的。将 $V_{C_0} \stackrel{k}{=} V_{C_1}$ 简化为 $B_{1,k-1,1}^{\uparrow,k-1} \stackrel{k-1}{=} M_1$ 后，我们可进一步将其简化为 $M_2 \stackrel{k-2}{=} B_{2,k-2,1}^\uparrow$ ：

$$\begin{aligned} M_1 &= \bigcup_{j=1\dots k} B_{j,k-j,1}^\uparrow \cup B_{0,k,1}^{\uparrow,\neg} \\ B_{1,k-1,1}^{\uparrow,k-1} &= B_{0,k-2,2}^\uparrow \cup B_{1,k-2,2}^\uparrow \cup D_{\{\}} \cup M_2^\neg \\ &\stackrel{k-2}{=} B_{0,k-2,2}^{\uparrow,\neg} \cup B_{1,k-2,2}^\uparrow \cup D_{\{\}} \cup M_2. \end{aligned}$$

根据引理 3.9 和 3.10，除 $B_{2,k-2,1}^\uparrow$ 和 M_2 外，所有子树都是 $(k-2)$ 等价的。

显然，我们可以继续以相同方式逐步减少 $V_{C_0} = V_{C_1}$ 直至 0。归纳步骤中，我们假设 $r < k$ 的

$V_{C_0} \triangleq V_{C_1} \Leftarrow B_{r,k-r,1}^{\uparrow,k-r} = M_r$ 成立，并证明

$$V_{C_0} \stackrel{k}{=} V_{C_1} \Leftarrow_{M_r} B_{r+1,k-r-1,1}^{\uparrow,k-r-1} \stackrel{k-r-1}{=} M_{r+1} \cup B_{r-1,k-r+1,1}^{\uparrow,\neg}$$

$$\begin{aligned} B_{r,k-r,1}^{\uparrow,k-r-1} &= \bigcup_{j=0\dots r} B_{j,k-r-1,2}^\uparrow \cup D_{\{\}} \cup M_{r+1}^\neg \\ &\stackrel{k-r-1}{=} \bigcup_{\substack{j=0\dots r \\ j \neq r-1}} B_{j,k-r-1,2}^\uparrow \cup B_{r-1,k-r-1,2}^{\uparrow,\neg} \\ &\quad \cup \bigcup_{j=r+2\dots k} B_{j,k-j,2}^\uparrow \cup M_{r+1}. \end{aligned}$$

除 M_{r+1} （或 $B_{r+1,k-r-1,1}^\uparrow$ ）外，根据引理 3.9 和 3.10，所有子树都是 $(k-r-1)$ 等价的。由于 M_{r+1} 和 $B_{r+1,k-r-1,1}^\uparrow$ 是唯一未立即匹配的子树，归纳步骤成立。对于 $r = k-1$ ，我们得到 $V_{C_0} \triangleq V_{C_1} \Leftarrow B_{k,0,1}^\uparrow = M_k$ ，由于 $B_{k,0,1}^\uparrow \stackrel{0}{=} M_k$ 成立，证明至此完成。 $\square$

注记。作为附带结果，定理 3.12 的证明凸显了 CT_k 中关键路径 $P = (\delta_1, \delta_2, \dots, \delta_k)$ 的根本重要性。沿路径 P 行进后，节点 $v \in C_0$ 的视图最终落在 C_0 相邻的叶簇中，并看到 δ_{i+1} 个邻居。沿相同路径，节点 $v' \in C_1$ 最终落在 C_0 中，并看到 $\sum_{j=0} \delta_j - 1$ 个邻居。这些视图无法

匹配。这种固有有不等性是定义 G_k 的根本原因：必须确保关键路径至少长 k 跳。

3.4 分析

在本小节中，我们推导 k 局部 MVC 算法近似比的下界。设 OPT 为 MVC 的最优解， ALG 为任意算法计算出的解。主要观察结果是，簇 C_0 和 C_1 中的相邻节点具有相同视图

因此，每个算法对两个簇中的节点一视同仁。结果， ALG 包含 C_0 中相当一部分节点，而最优解完全由 C_1 中的节点覆盖 C_0 与 C_1 之间的边。

引理3.13. 设 ALG 为任意运行至多 k 轮的分布式（随机化）顶点覆盖算法的解。当应用于第3.2小节构造的 G_k 时，在最坏情况下（期望意义上）， ALG 至少包含 C_0 中一半的节点。

证明。设 $v_0 \in C_0$ 和 $v_1 \in C_1$ 是来自 C_0 和 C_1 的两个任意相邻节点。我们首先对确定性算法证明该引理。给定节点 v 是否进入顶点覆盖仅取决于拓扑 $\mathcal{T}_{v,k}$ 和标记 $\mathcal{L}(\mathcal{T}_{v,k})$ 。假设图的标记是均匀随机选择的。进一步，设 p_0^A 和 p_1^A 分别表示当确定性算法 A 在随机选择的标记上运行时， v_0 和 v_1 最终进入顶点覆盖的概率。根据定理3.12， v_0 和 v_1 看到相同的拓扑，即 $\mathcal{T}_{v_0,k} = \mathcal{T}_{v_1,k}$ 。根据我们的标记选择， v_0 和 v_1 在标记 $\mathcal{L}(\mathcal{T}_{v_0,k})$ 和 $\mathcal{L}(\mathcal{T}_{v_1,k})$ 上也看到相同的分布。因此可得 $p_0^A = p_1^A$ 。

我们选择 v_0 和 v_1 使得它们在 G_k 中是邻居。为了获得可行的顶点覆盖，两个节点中至少有一个必须在其中。这意味着 $p_0^A + p_1^A \geq 1$ ，因此 $p_0^A = p_1^A \geq 1/2$ 。换句话说，对于 C_0 中的所有节点，最终进入顶点覆盖的概率至少为 $1/2$ 。因此，根据期望的线性性质， C_0 中至少一半的节点被算法 A 选中。因此，对于每个确定性算法 A ，至少存在一个标记，使得 C_0 中至少一半的节点在顶点覆盖中。²

对于随机化算法的论证现在利用姚氏最小最大原理变得直接。随机化算法选择的期望节点数不能小于最优确定性算法在任意选择的标记分布上选择的期望节点数。

□

引理3.13给出了任何 k -局部MVC算法所选节点数的下界。特别地，我们有 $\mathbb{E}[|ALG|] \geq |C_0|/2 = n_0/2$ 。我们不知道 OPT ，但由于簇 C_0 的节点对于获得可行顶点覆盖并非必要，最优解受限于 $|OPT| \leq n - n_0$ 。在下文中，我们定义

$$\delta_i := \delta^i, \forall i \in \{0, \dots, k+1\} \quad (4)$$

对于某个值 δ 。

引理3.14. 如果 $k+1 < \delta$ ，则 G_k 的节点数 n 为

$$n \leq n_0 \left(1 + \frac{k+1}{\delta - (k+1)} \right).$$

证明。 C_0 中有 n_0 个节点。根据(4)，每个簇的节点数每增加一层减少因子 δ 。因此，第 l 层上的簇包含 n_0/δ^l 个节点。

² 实际上，由于顶点覆盖中最多有 $|C_0|$ 个这样的节点，因此在至少 $1/3$ 的标记中，该数量超过 $|C_0|/2$ 。

根据 CT_k 的定义，每个簇在更高层级上最多有 $k+1$ 个相邻簇。因此，层级 l 上的节点数 n_l 的上界为

$$n_l \leq (k+1)^l \cdot \frac{n_0}{\delta^l}.$$

对所有层级 l 求和，并将该和视为几何级数，我们得到

$$\begin{aligned} n &\leq n_0 \cdot \sum_{i=0}^{k+1} \left(\frac{k+1}{\delta} \right)^i \leq n_0 \cdot \sum_{i=0}^{\infty} \left(\frac{k+1}{\delta} \right)^i \\ &= n_0 + n_0 \left(\frac{k+1}{\delta} \right) \left(\frac{1}{1 - \frac{k+1}{\delta}} \right) \\ &= n_0 \left(1 + \frac{k+1}{\delta - (k+1)} \right). \end{aligned}$$

□

仍需确定 δ 与 n_0 之间的关系，使得 G_k 能按第 3.2 节所述实现。在该节中，具有大围长的 G_k 的构造基于一个较小的实例 G'_k ，其中围长无关紧要。利用 (4)（即 $\delta_i := \delta^i$ ），我们现在可以解决这个遗留问题，并描述如何获得 G'_k 。每个簇的节点数在 CT_k 的每一层级上减少一个因子 δ 。包含 C_0, CT_k 由 $k+2$ 个层级组成。叶簇内部的最大邻居数为 δ^k 。因此，我们可以将最外层 $k+1$ 上的簇大小设为 δ^k 。这意味着层级 l 上的簇大小为 δ^{2k+1-l} 。特别地， G'_k 中层级 0 上的 C'_0 的大小为 $n'_0 = \delta^{2k+1}$ 。设 C_i 和 C_j 为两个相邻簇，其中 $\ell(C_i, C_j) = (\delta^i, \delta^{i+1})$ 。 C_i 和 C_j 可以简单地通过所需数量的完全二分图 $K_{\delta^i, \delta^{i+1}}$ 连接。

如果我们假设 $k+1 \leq \delta/2$ ，则由引理 3.14 可得 $n \leq 2n_0$ 。应用第 3.2 节的构造，我们得到 $n_0 \leq n'_0 \cdot \langle n' \rangle^{2k-5}$ ，其中 $\langle n' \rangle$ 表示大于或等于 n' 的最小素数幂，即 $\langle n' \rangle < 4n'_0$ 。综合所有，我们得到

$$n_0 \leq (4n'_0)^{2k-4} \leq 4^{2k-4} \delta^{4k^2}. \quad (5)$$

定理 3.15. 存在图 G ，使得在 k 轮通信中， G 上最小顶点覆盖问题的每个分布式算法的近似比至少为

$$\Omega\left(\frac{n^{c/k^2}}{k}\right) \quad \text{and} \quad \Omega\left(\frac{\Delta^{1/k}}{k}\right)$$

对于某个常数 $c \geq 1/4$ ，其中 n 和 Δ 分别表示 G 中的节点数和最高度数。

证明。由于不等式 (5)，我们可以选择 $\delta \geq 4^{-1/(2k)} n_0^{1/(4k^2)}$ 。最后，利用引理 3.13 和 3.14，近似比 α 至少为

$$\begin{aligned} \alpha &\geq \frac{n_0/2}{n - n_0} \geq \frac{n_0/2 \cdot \delta/2}{n_0 \cdot (k+1)} = \frac{\delta}{4(k+1)} \\ &\geq \frac{(n/2)^{1/(4k^2)}}{4^{1+1/(2k)}(k+1)} \in \Omega\left(\frac{n^{1/(4k^2)}}{k}\right). \end{aligned}$$

第二个下界由 $\Delta = \delta^{k+1}$ 得出。

□

定理3.16. 为了获得多对数甚至常数近似比, 每个用于MVC问题的分布式算法至少需要 $\Omega\left(\sqrt{\frac{\log n}{\log \log n}}\right)$ 和

$\Omega\left(\frac{\log \Delta}{\log \log \Delta}\right)$ 轮通信。

证明. 我们为任意常数 $\beta > 0$ 设置 $k = \beta \sqrt{\log n / \log \log n}$ 。将其代入定理3.15的第一个下界时, 得到以下近似比 α :

$$\alpha \geq \gamma n^{\frac{c \log \log n}{\beta^2 \log n}} \cdot \frac{1}{\beta} \sqrt{\frac{\log \log n}{\log n}}$$

其中 γ 是 Ω 记号中的隐藏常数。对于 α 的对数, 我们得到

$$\begin{aligned} \log \alpha &\geq \frac{c \log \log n}{\beta^2 \log n} \cdot \log n - \frac{1}{2} \cdot \log \log n - \log \beta \\ &= \left(\frac{c}{\beta^2} - \frac{1}{2} \right) \cdot \log \log n - \log \beta. \end{aligned}$$

因此

$$\alpha \in \Omega\left(\log(n)^{\left(\frac{c}{\beta^2} - \frac{1}{2}\right)}\right).$$

通过选择合适的 β , 我们可以确定上述表达式的指数。对于每个多对数项 $\alpha(n)$, 存在一个常数 β 使得上述表达式至少为 $\alpha(n)$, 因此定理的第一个下界成立。

第二个下界通过设置 $k = \beta \log \Delta / \log \log \Delta$ 进行类似计算得出。□

记注. 通过定义 $\delta_i := \delta^i, i \in \{0, \dots, k\}$ 和 $\delta_{k+1} := \delta^{k+1/2}$ (而非 δ^{k+1}), 我们得到稍强的近似下界

$$\Omega n^{c/k^2} - k \quad \text{and} \quad \Omega \Delta^{c'/k} - k. \quad (6)$$

(6)中的下界显然不足以改进定理3.16的结果。

4. 归约

利用顶点覆盖的下界, 我们可以得到其他几个经典图问题的下界。在本节中, 我们给出最大匹配和最大独立集构造以及最小支配集近似的时间下界。

图 G 的最大匹配 (MM) 是不共享公共端点的边的最大集合。因此, MM是 G 的一组不相邻边, 使得 $E(G) \setminus MM$ 中的所有边都与MM中的某条边共享一个公共端点。最大独立集 (MIS) 是不相邻节点的最大集合, 即不在MIS中的所有节点都与MIS中的某个节点相邻。已知的MM或MIS分布式计算最佳下界是 $\Omega(\log^* n)$, 该下界适用于环[15]。基于定理3.16, 我们得到以下更强的下界。

定理4.1. 存在图 G , 在其上每个分布式 (可能随机化) 算法需要时间

$$\Omega\left(\sqrt{\frac{\log n}{\log \log n}}\right) \quad \text{and} \quad \Omega\left(\frac{\log \Delta}{\log \log \Delta}\right)$$

来计算最大匹配。同样的下界也适用于最大独立集的构造。

证明. 众所周知, MM边的所有端点集合构成MVC的2-近似。这个简单的2-近似算法通常归功于Gavril和Yannakakis。因此, MM构造的下界直接来自定理3.16。

对于MIS问题, 考虑 G_k 的线图 $L(G_k)$ 。 G 的线图 $L(G)$ 的节点是 G 的边。每当 G 中两条对应的边关联到同一个节点时, $L(G)$ 中的两个节点就由一条边连接。图 G 上的MM问题等价于 $L(G)$ 上的MIS问题。此外, 如果真实网络图是 G, k , 则 $L(G)$ 上的通信轮次可以在 G 上的 $k + O(1)$ 通信轮次中模拟。因此, 在 $L(G_k)$ 上计算MIS的时间 t 与在 G_k 上计算MM的时间 t' 仅相差一个常数 $t \geq t' - O(1)$ 。设 n' 和 Δ' 分别表示 G_k 的节点数和最大度数。 $L(G_k)$ 的节点数 n 小于 $n'^2/2$, G_k 的最大度数 Δ 小于 $2\Delta'$ 。由于 n' 仅以 $\log n'$ 的形式出现, 2的幂次不会造成影响, 定理成立 ($\log n = \Theta(\log n')$)。 □

我们通过考虑最小支配集 (MDS) 问题的近似的来结束本节。支配集 S 是图 G 节点的一个子集, 使得 G 的所有节点要么在 S 中, 要么在 S 中有一个邻居。在非分布式环境中, MDS等价于一般的最小集合覆盖问题³, 而MVC是集合覆盖的一个特例, 可以近似得更好。因此, 在分布式环境中, MDS严格比MVC更难也就不足为奇了。下面, 我们展示这一直观事实可以被形式化。

定理4.2. 存在图 G , 使得在 k 通信轮次内, G 上最小支配集问题的每个分布式算法的近似比至少为

$$\Omega\left(\frac{n^{c/k^2}}{k}\right) \quad \text{and} \quad \Omega\left(\frac{\Delta^{1/k}}{k}\right)$$

对于某个常数 c , 其中 n 和 Δ 分别表示 G 中的节点数和最高度数。

证明. 我们证明每个MVC实例都可以视为具有相同局部性的MDS实例。设 $G' = (V', E')$ 是一个我们想要解决MVC的图。我们按如下方式构造相应的支配集图 $G = (V, E)$ 。对于 G' 中的每个节点和每条边, 在 G 中都有一个节点。我们将对应于节点 $v' \in V'$ 的节点称为 $v_n \in V$ 节点, 将对应于边 $e' \in E'$ 的节点称为 $v_e \in V$ 节点。两个 n 节点之间有一条边当且仅当它们在 G' 中相邻。一个 n 节点 v_n 和一个 e 节点 v_e 之间有一条边当且仅当对应的节点和边在 G' 中关联。两个 e 节点之间没有边。显然, G' 和 G 的局部性相同, 即在一个图上进行的 k 轮通信可以在另一个图上用 $k + O(1)$ 轮模拟。设 C 是 G' 的一个可行顶点覆盖。我们声称 G 中的所有节点

³ 在两个方向上存在保持近似性的归约。

G 中对应于 C 中节点的节点构成 G 上的一个有效支配集。根据定义, 所有 e 节点都被覆盖。 G 中剩余的节点也被覆盖, 因为对于给定图, 有效顶点覆盖也是有效支配集。因此, G 上的最优支配集至多与 G' 上的最优顶点覆盖一样大。另一个方向也存在变换。设 D 是 G 上的一个有效支配集。如果 D 包含一个 e 节点 v_e , 我们可以将 v_e 替换为其两个邻居之一。 D 的大小保持不变, 并且被 v_e 覆盖 (支配) 的所有三个节点仍然被覆盖。由此, 我们得到一个与 D 大小相同且仅由 n 节点组成的支配集 D' 。因为 D' 支配所有 e 节点, G' 中对应于 D' 的节点构成一个有效顶点覆盖。因此, G 上的MDS与 G' 上的MVC难度完全相同, 该定理由定理3.15得出。

推论4.3。为了获得最小支配集的多对数或常数近似比, 存在一些图, 使得每个分布式算法都需要时间

$$\Omega\left(\sqrt{\frac{\log n}{\log \log n}}\right) \text{ and } \Omega\left(\frac{\log \Delta}{\log \log \Delta}\right).$$

证明。该推论是定理4.2和定理3.16证明的直接结果。

注。注意在上述推论中, 我们给出了常数MDS近似的时间下界, 尽管已有研究表明MDS的近似比不能优于 $\ln \Delta$, 除非 $\text{NP} \subseteq \text{DTIME}(n^{O(\log \log n)})$ [5]。然而, 由于在我们的模型中局部计算是免费的, 理论上仍有可能获得MDS的常数因子近似。

5. 结论

随着分布式系统规模不断扩大, 设计无需维护网络完整信息的算法变得愈发关键。遗憾的是, 除少数显著例外[15], 几乎未有坚实成果能阐明基于局部性方法的理论可能性与局限性。我们已证明局部性对若干分布式问题的近似性与可计算性施加了限制。与相应上界相比, 部分下界近乎紧致。我们期望并相信本文给出的多种下界将有助于改善这一现状。

6. 参考文献

- [1] Y. Afek, S. Kutten, and M. Yung. 局部检测范式及其在自稳定中的应用. 理论计算机科学, 186(1-2):199-229, 1997.
- [2] R. Cole and U. Vishkin. 确定性抛硬币及其在最优并行列表排序中的应用. 信息与控制, 70(1):32-53, 1986.
- [3] M. Elkin. 一种更快的分布式最小生成树构建协议. 见第 15th 届ACM-SIAM离散算法研讨会(SODA)论文集, 第359-368页, 2004.
- [4] M. Elkin. 分布式最小生成树问题时间-近似权衡的无条件下界. 见第 36th 届ACM计算理论研讨会(STOC)论文集, 2004.
- [5] U. Feige. 集合覆盖近似的 $\ln n$ 阈值. ACM期刊 (JACM), 45(4):634-652, 1998.
- [6] F. Fich and E. Ruppert. 分布式计算的数百个不可能性结果. 分布式计算, 16(2-3):121-163, 2003.
- [7] M. J. Fischer, N. A. Lynch, and M. S. Paterson. 单故障进程下分布式共识的不可能性. ACM期刊, 32(2):374-382, 1985.
- [8] A. Israeli and A. Itai. 一种快速简单的随机并行最大匹配算法. 信息处理快报, 22:77-80, 1986.
- [9] F. Kuhn and R. Wattenhofer. 常数时间分布式支配集近似. 见第 22nd 届ACM分布式计算原理年会(PODC)论文集, 第25-32页, 2003.
- [10] F. Kuhn and R. Wattenhofer. 分布式组合优化. 技术报告426, 苏黎世联邦理工学院计算机科学系, 2003.
- [11] E. Kushilevitz and Y. Mansour. 无线网络广播的 $\Omega(D \log(N/D))$ 下界. SIAM计算期刊, 27(3):702-712, 1998年6月.
- [12] L. Lamport, R. Shostak, and M. Pease. 拜占庭将军问题. ACM编程语言与系统汇刊, 4(3):382-401, 1982.
- [13] F. Lazebnik and V. A. Ustimenko. 具有任意大围长和大尺寸图的显式构造. 离散应用数学, 60(1-3):275-284, 1995.
- [14] F. Lazebnik, V. A. Ustimenko, and A. J. Woldar. 高围密图的新系列. 美国数学会通报(新系列), 32(1):73-79, 1995.
- [15] N. Linial. 分布式图算法中的局部性. SIAM Journal on Computing, 21(1):193-201, 1992.
- [16] Z. Lotker, B. Patt-Shamir, 和 D. Peleg. 恒定直径图的分布式最小生成树. 载于第 20th 届ACM分布式计算原理年度研讨会(PODC)论文集, 第63-71页, 2001.
- [17] M. Luby. 最大独立集问题的一种简单并行算法. SIAM Journal on Computing, 15:1036-1053, 1986.
- [18] M. Naor 和 L. Stockmeyer. 局部可计算什么? 载于第 25th 届ACM计算理论年度研讨会(STOC)论文集, 第184-193页, 1993.
- [19] D. Peleg. 分布式计算: 一种局部性敏感方法. SIAM, 2000.
- [20] D. Peleg 和 V. Rubinovich. 分布式最小权重生成树构建时间复杂度的近紧下界. SIAM Journal on Computing, 30(5):1427-1442, 2000.

APPENDIX

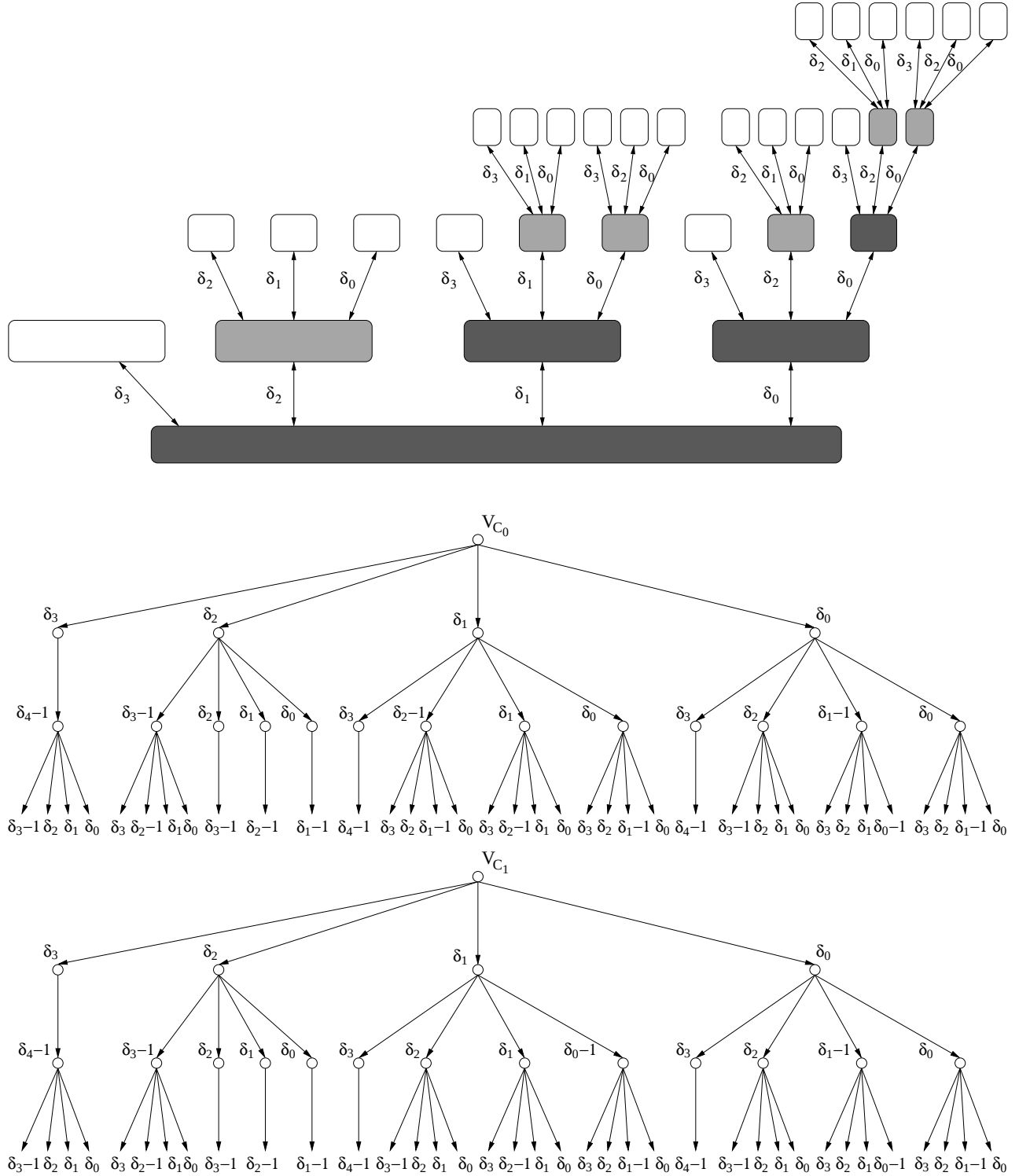


图3: 聚类树 CT_3 以及 C_0 和 C_1 中节点对应的视图树。聚类树 CT_1 和 CT_2 分别以深色和浅色阴影表示。聚类树弧上的标签表示低层集群节点在相邻高层集群中的邻居数量。反向链接的标签已省略。在视图树中, 标有 δ_i 的弧代表 δ_i 条边, 全部连接到相同的子树。